```
"""
STEP 1: Install and Import Required Libraries

This cell imports all necessary libraries required for:
- Model loading (Transformers)
- Dataset handling (Datasets)
- Parameter-efficient fine-tuning (PEFT with LoRA)
- Deep learning backend (PyTorch)
"""

# Install packages (run once in Colab)
# !pip install torch transformers datasets peft accelerate

# Import PyTorch for tensor operations
import torch

# Import model and tokenizer utilities from Hugging Face
from transformers import AutoModelForSequenceClassification, AutoTokenizer

# Import dataset loader
from datasets import load_dataset

# Import LoRA configuration and PEFT model wrapper
from peft import LoraConfig, get_peft_model

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

```
"""
STEP 2: Load Pre-trained Model and Tokenizer

- Loads a transformer model (DistilBERT)
- Loads tokenizer to convert text into tokens
- Prepares model for classification task (binary sentiment)
"""

# Define model name (pre-trained model)
model_name = "distilbert-base-uncased"

# Load tokenizer
tokenizer = AutoTokenizer.from_pretrained(model_name)

# Load pre-trained model with 2 output labels (positive/negative)
model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=2
)

print("Model and tokenizer loaded successfully!")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingfac
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datas
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable h
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hu
config.json: 100%  483/483 [00:00<00:00, 17.1kB/s]

tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 1.47kB/s]

vocab.txt: 232k/? [00:00<00:00, 2.52MB/s]

tokenizer.json: 466k/? [00:00<00:00, 5.58MB/s]

model.safetensors: 100%  268M/268M [00:02<00:00, 288MB/s]

Loading weights: 100%  100/100 [00:00<00:00, 332.25it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.w

DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key                      | Status     |
-------------------------+------------+-
vocab_projector.bias     | UNEXPECTED |
vocab_layer_norm.bias    | UNEXPECTED |
vocab_transform.weight   | UNEXPECTED |
vocab_layer_norm.weight  | UNEXPECTED |
vocab_transform.bias     | UNEXPECTED |
pre_classifier.bias      | MISSING    |
classifier.bias          | MISSING    |
classifier.weight        | MISSING    |
pre_classifier.weight    | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expe
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider tra
Model and tokenizer loaded successfully!
```

```
"""
STEP 3: Load and Preprocess Dataset

- Loads IMDB dataset
- Tokenizes text into model-understandable format
- Splits into training and testing subsets
"""

# Load dataset (IMDB sentiment dataset)
dataset = load_dataset("imdb")

# Define tokenization function
def tokenize_function(example):
    """
    Converts raw text into tokenized format.

    Parameters:
    example (dict): A dictionary containing text data

    Returns:
    dict: Tokenized output with padding and truncation
    """
    return tokenizer(example["text"], truncation=True, padding="max_length")

# Apply tokenization to entire dataset
tokenized_dataset = dataset.map(tokenize_function, batched=True)

# Select small subset for faster training
train_dataset = tokenized_dataset["train"].shuffle(seed=42).select(range(500))
test_dataset = tokenized_dataset["test"].shuffle(seed=42).select(range(200))
```

```
print("Dataset loaded and tokenized successfully!")
```

```
README.md:        7.81k/? [00:00<00:00, 465kB/s]
plain_text/train-00000-of-00001.parquet: 100%  21.0M/21.0M [00:00<00:00, 90.6MB/s]
plain_text/test-00000-of-00001.parquet: 100%  20.5M/20.5M [00:00<00:00, 102MB/s]
plain_text/unsupervised-00000-of-00001.p(…): 100%  42.0M/42.0M [00:00<00:00, 209MB/s]
Generating train split: 100%  25000/25000 [00:00<00:00, 87024.28 examples/s]
Generating test split: 100%  25000/25000 [00:00<00:00, 96809.01 examples/s]
Generating unsupervised split: 100%  50000/50000 [00:00<00:00, 121798.66 examples/s]
Map: 100%  25000/25000 [00:30<00:00, 879.11 examples/s]
Map: 100%  25000/25000 [00:31<00:00, 703.45 examples/s]
Map: 100%  50000/50000 [01:09<00:00, 772.73 examples/s]
Dataset loaded and tokenized successfully!
```

```
"""
STEP 4: Configure LoRA Adapter

- Defines LoRA parameters such as rank, alpha, dropout
- Specifies which layers to modify (attention layers)
"""

# Create LoRA configuration
lora_config = LoraConfig(
    r=8,  # Rank of low-rank matrices
    lora_alpha=16,  # Scaling factor
    target_modules=["q_lin", "v_lin"],  # Target attention layers
    lora_dropout=0.1,  # Dropout for regularization
    bias="none",  # No bias training
    task_type="SEQ_CLS"  # Task type: Sequence Classification
)

print("LoRA configuration created!")
```

```
LoRA configuration created!
```

```
"""
STEP 5: Apply PEFT Model

- Attaches LoRA adapters to base model
- Only small adapter parameters become trainable
- Base model remains frozen
"""

# Apply LoRA to the model
peft_model = get_peft_model(model, lora_config)

# Print number of trainable parameters
peft_model.print_trainable_parameters()
```

```
WARNING:torchao.kernel.intmm:Warning: Detected no triton, on systems without Triton certain kernel
trainable params: 739,586 || all params: 67,694,596 || trainable%: 1.0925
```

```
"""
STEP 6: Train the PEFT Model

- Defines training configuration
- Uses Hugging Face Trainer API
- Trains only LoRA parameters (efficient training)
"""

# Import Trainer and TrainingArguments
from transformers import TrainingArguments, Trainer

# Define training settings
training_args = TrainingArguments(
    output_dir="./results",  # Output directory
    learning_rate=2e-4,  # Learning rate
    per_device_train_batch_size=8,  # Batch size
    num_train_epochs=1,  # Number of epochs
    logging_steps=10,  # Log frequency
    save_strategy="no"  # Do not save checkpoints
)

# Initialize Trainer
trainer = Trainer(
    model=peft_model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset
)

# Start training
trainer.train()

print("Training completed!")
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memo
  super().__init__(loader)
```

[58/63 17:49 < 01:35, 0.05 it/s, Epoch 0.90/1]

Step	Training Loss
10	0.570096
20	0.534809
30	0.530155
40	0.394280
50	0.388053

[63/63 19:32, Epoch 1/1]

Step	Training Loss
10	0.570096
20	0.534809
30	0.530155
40	0.394280
50	0.388053
60	0.354060

```
Training completed!
```

```
# Set model to evaluation mode
peft_model.eval()

# Evaluate model using test dataset
```

```
results = trainer.evaluate(eval_dataset=test_dataset)
print("Evaluation Results:", results)

# Device setup
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
peft_model.to(device)

# Input text
text = "The movie was fantastic and inspiring!"

# Tokenize
inputs = tokenizer(text, return_tensors="pt", truncation=True, padding=True)

# Move to device
inputs = {k: v.to(device) for k, v in inputs.items()}

# Prediction
with torch.no_grad():
    outputs = peft_model(**inputs)

prediction = torch.argmax(outputs.logits, dim=1)

print("Predicted label:", prediction.item())

# Optional label mapping
labels = ["Negative", "Positive"]
print("Predicted sentiment:", labels[prediction.item()])
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memo
  super().__init__(loader)
```

[25/25 03:01]

```
Evaluation Results: {'eval_loss': 0.3857247531414032, 'eval_runtime': 194.8409, 'eval_samples_per_
Predicted label: 1
Predicted sentiment: Positive
```